

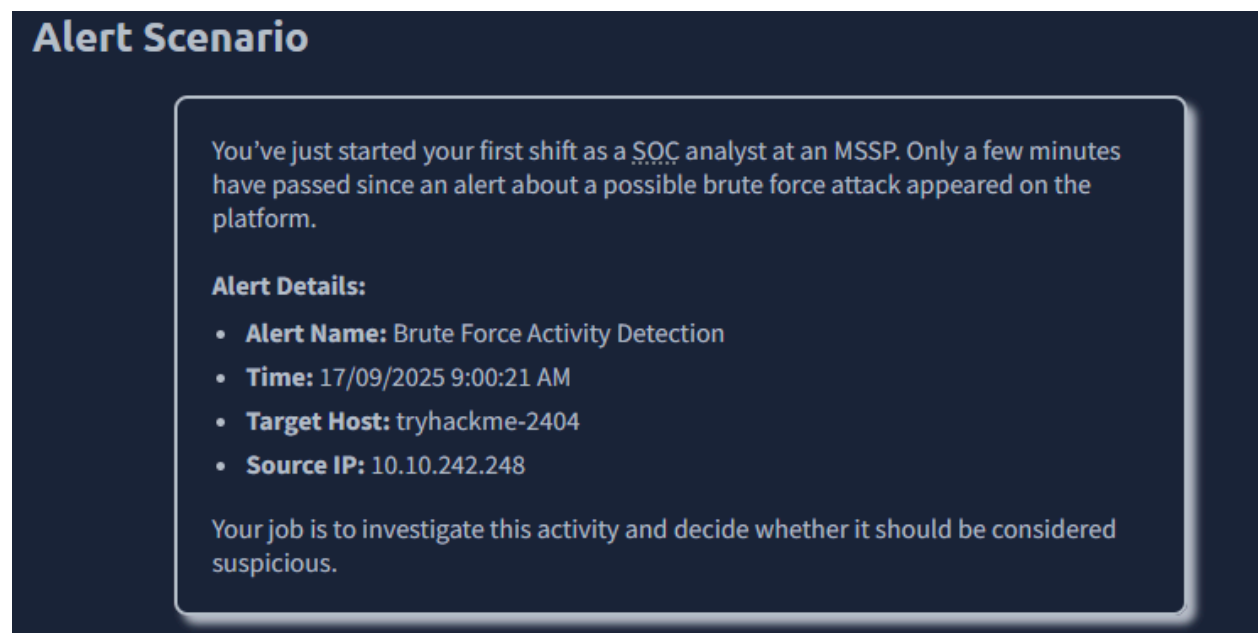
SOC/L1 -13.2. Alert Triage With Splunk

Task 1 - Introduction

As a SOC analyst, it's important to be able to investigate different types of suspicious activity across a variety of assets in the environment.

Task 2 - Initial Access Alert

The first one will take place on a Linux host, and you will uncover what exactly happens there during our investigation. Representing how adversaries gain their **first foothold** in a target environment. It includes nine primary techniques, with **phishing (T1566)** being the most common vector, accounting for over 40% of breaches.

A screenshot of a dark-themed interface titled "Alert Scenario". The text describes a scenario where a SOC analyst at an MSSP receives an alert about a possible brute force attack. It lists alert details: Alert Name (Brute Force Activity Detection), Time (17/09/2025 9:00:21 AM), Target Host (tryhackme-2404), and Source IP (10.10.242.248). The task is to investigate this activity and decide if it's suspicious.

Alert Scenario

You've just started your first shift as a SOC analyst at an MSSP. Only a few minutes have passed since an alert about a possible brute force attack appeared on the platform.

Alert Details:

- **Alert Name:** Brute Force Activity Detection
- **Time:** 17/09/2025 9:00:21 AM
- **Target Host:** tryhackme-2404
- **Source IP:** 10.10.242.248

Your job is to investigate this activity and decide whether it should be considered suspicious.

```
index="linux-alert"
```

```
data > data&time range > 17/09/2025 9:00:21 AM
```

```
sourcetype > linux_source  
source > auth.log
```

This query will search for both successful and failed login attempts, as well as events related to invalid users, which are commonly observed when an attacker is attempting to enumerate accounts before starting brute force attacks on a specific user.

```
index="linux-alert" sourcetype="linux_secure" 10.10.242.248  
| search "Accepted password for" OR "Failed password for" OR "Invalid user"  
| sort + _time
```

Answer the questions below

| How many failed login attempts were made on the user john.smith? 500

john.smith shows a significantly high number of 503 attempts, which is a clear indicator of brute force activity. But we need only failed login, no need to write "accepted" or "invalid user".

```
index="linux-alert" sourcetype="linux_secure" 10.10.242.248  
| rex field=_raw "^{d{4}-{d{2}-{d{2}}T[^\s]+\s+(?<log_hostname>\S+)"  
| rex field=_raw "sshd\[.{d+}\]:s*(?<action>Failed)\s+\S+\s+for(?: invalid user)? (?  
<username>\S+) from (?<src_ip>.{d{1,3}}(?:\.{d{1,3}}){3})"  
| eval process="sshd"  
| stats count values(action) values(src_ip) as src_ip values(log_hostname) as  
hostname values(process) as process by username
```

| What was the duration of the brute force attack in minutes?

```
index="linux-alert" sourcetype="linux_secure" 10.10.242.248  
| search "Failed password for" OR "Failed" OR "Accepted"  
| sort + _time (chose user , check alert time from first to last)
```

| What username was the attacker able to privilege escalate to?

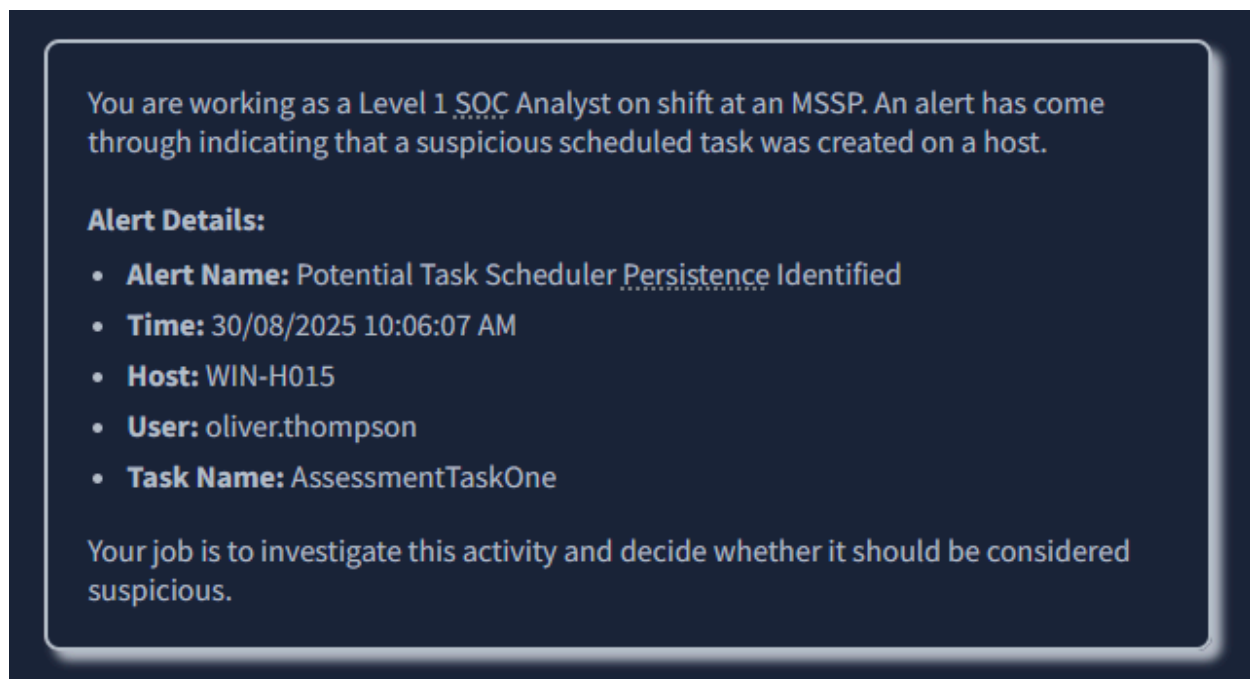
```
index="linux-alert" sourcetype="linux_secure" "sudo" "su"
```

| What is the name of the user account created by the attacker for persistence?

```
index="linux-alert" "adduser"  
filter user, time, timestamp, process  
host > tryhackme-2404 > user added (SYSTEM-UTM)
```

Task 2- Persistent Alert

The second scenario will focus on activity in a Windows environment, which SOC analysts come across on a regular basis. That's why knowing how to carry out the analysis is really important for you. A **persistent alert** or notification is **an alert that remains visible on a device or interface until the user explicitly acknowledges, dismisses, or takes action on it, ensuring critical information is not missed**. Unlike temporary notifications that fade after a few seconds, persistent alerts demand attention for time-sensitive or essential tasks,



```
index="win-alert" Scheduler  
check alert > Task Scheduler Configuration tool
```

Answer the questions below

| What is the ProcessId of the process that created this malicious task?

```
index="win-alert" EventCode=4698 AssessmentTaskOne  
check > Pid on event table > check ClientProcessId > 5816
```

What is the name of the parent process for the process that created this malicious task?

```
index="win-alert" ProcessId=4128  
check > commandLine > cmd.exe
```

Which local group did the attacker enumerate during discovery?

```
index="win-alert" ProcessId=4128  
| table _time ParentCommandLine CommandLine  
net localgroup Administrators
```

What is the name of the workstation from which the Threat Actor logged into this host?

```
index="win-alert" EventCode=4624 (log into related to eventCode 4624)  
| table _time Workstation_Name process authentication_method user  
| sort + _time (get > )
```

Task 3 - Web Shell Alert

The last scenario will focus on activity in a vulnerable web server. We'll be investigating possible web shell exploitation. **Web shells** are malicious scripts uploaded to a web server that enable **remote administration** and unauthorized access, often leading to wider network compromise. They are commonly written in **PHP, ASP, or Python** and are delivered via vulnerabilities such as **SQL injection, file upload weaknesses, or CMS exploits**.

Your shift as an L1 SOC analyst continues, and you've now received the next alert that needs to be investigated. This time, the activity is related to the web.

Alert Details:

- **Alert Name:** Potential Web Shell Upload Detected
- **Time:** 14/09/2025 09:31:51 AM
- **Resource:** <http://web.trywinme.thm>
- **Suspicious IP:** 171.251.232.40

Your job is to investigate this activity and decide whether it should be considered suspicious.

```
index= "web-alert"
```

```
clientip="171.251.232.40"
```

```
index=web-alert 171.251.232.40 b374k.
```

```
| table _time clientip useragent uri_path referer referer_domain method status  
| sort + _time (got web shell file, b374k.php)
```

```
index=web-alert "b374k.php"
```

```
url_query > file=b374k.php
```

Got more info about web shell (b374k) in Github

Answer the questions below

| What time did the brute-force activity using Hydra begin?

```
index=web-alert 171.251.232.40
```

```
| table _time clientip useragent uri_path method status  
| sort + _time (got 2025-09-14 21:20:27)
```

| Which user agent did the attacker use when interacting with the web shell?

(same query above)

| What was the number of requests made by the attacker to the server via the web shell?

Request = post (same query above, number of post 4)